

The Future of Developers: More Paths In, Not Out

The next generation of developers won't come from computer science alone — they'll come from philosophy, design, history, and business. And we'll need more of them than ever.

The Misconception

There's a common perception that developers are on the way out — that AI and automation will push engineers out of the profession and into unrelated careers. A recent LinkedIn exchange crystallised this: someone implied that "different paths for developers" meant paths *away* from development, into gardening or whatever else.

That's backwards.

The different paths I'm talking about are paths **into** development, from other professions. And we're going to need *more* developers and engineers, not fewer.

What Is a Developer, Anyway?

There's a longstanding argument that software development isn't really engineering. And honestly, there's some truth to it — the only people arguably doing real engineering in most software shops are QA and testers, because everyone else is practising something closer to art than science.

Being a developer, at its core, is about **solving problems**. It's about breaking things into small pieces, iterating, refactoring. It's a craft. The Wardley Maps evolution, the non-engineering capabilities — that's what makes a good developer. Not syntax. Not frameworks.

The Vibe Coding Explosion

Vibe coding is changing the game, but not in the way people think.

Consider this: **management has been "vibe coding" their entire careers**. They've always had the intent and the vision — it was just painfully slow to iterate. Now they can iterate fast. Business owners and the people software exists to serve can now create genuinely good briefs — even working prototypes.

Excel was the original vibe coding tool. What we're seeing now is that same explosion of productivity, but orders of magnitude more powerful. Mini tools, custom solutions, rapid iteration.

This means we're going to move away from one-size-fits-all SaaS applications that solve 80% of the problem and make the remaining 20% even harder. The era of mass-market software with terrible user satisfaction is ending.

Coding Is the Easy Part

Here's the thing people miss: **coding is the easy part**. Infrastructure, reliability, security, all the non-functional requirements — that's the hard part.

Look at Anthropic itself — they've had their systems going up and down like a yo-yo. They have AI-coded developers. AI coding everywhere. But building systems that are robust, that work in production, that scale, that don't fall apart? That's hard. That's where real engineering lives.

This is the handover problem. Vibe coding gets you from zero to prototype. But getting from prototype to production-grade? That requires craft, discipline, and deep engineering skill.

Jevons Paradox Applied to Software

When something becomes more accessible, you don't reduce demand — you **increase** it. That's Jevons Paradox, and it applies directly here.

More people able to create software means more software being created, which means more complexity, more integration challenges, more need for people who can think systematically about how it all fits together. The demand for real developers and engineers doesn't shrink. It explodes.

Every team in every business should have developers — I've been saying this for years. In the past, teams were already producing software but with no control, getting the worst of both worlds. The few businesses that embedded engineers in every team succeeded spectacularly but were never copied. Now, that model isn't optional anymore.

The Paths Are Into Development, Not Out of It

The pioneers who built the first computers, created Apple, built the visual effects at Industrial Light & Magic — they weren't "developers" in the traditional sense. They were developing, writing software, doing engineering. But the label didn't exist yet because they were inventing the field.

My core premise: **the next generation of developers will come from outside traditional computer science**. They'll come from:

- **Philosophy** — bringing critical thinking and first-principles reasoning
- **Design** — bringing empathy, user experience, and human-centred problem solving
- **History** — bringing pattern recognition and an understanding of how systems evolve
- **Business** — bringing process understanding and the ability to map software to real value
- **Mechanical engineering, civil engineering, chemistry** — bringing disciplined engineering practice
- **Even acting and the humanities** — bringing communication, perspective-taking, and creativity

Brett Victor made this point brilliantly in his talk where he role-played as an IBM employee looking back from the future, showing how little innovation the industry actually produced — how we descended into dogma, where people don't just fail to find new solutions, they stop even *considering* that different solutions exist.

Two Emerging Archetypes

I see the developer role splitting into two complementary paths:

The Business Developer — over-indexed on understanding business processes, user empathy, mapping experience to delivery, and shipping. These are the people who bridge the gap between what the organisation needs and what gets built.

The Craft Developer — focused on reviewing code, ensuring rock-solid quality, reliability, security. These are the ones who make sure what gets released actually works at scale. The engineers in the truest sense.

Both are essential. Both are developers. They just operate at different points in the value chain.

Every Team Needs This

The conclusion is straightforward: every team in the business will need these professionals. Maybe we need a new name for it. But I think "developer" and "engineer" still work — we just need to expand our understanding of what those words mean.

The future isn't developers leaving the profession. It's the profession opening its doors wider than ever.